# Algoritmos Distribuidos

# Terminación

**Andrés Sanoja**
**Claudia León**
Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Febrero 2022



# Objetivos

- Comprender y describir el problema de terminación en Aplicaciones Distribuidas
- Comparar diferentes soluciones para la detección de la terminación de una aplicación distribuida.

# Detección de Terminación

Se dice que una aplicación distribuida **ha terminado** cuando alcanza un estado global o **configuración terminal**:  Una configuración en la que no existen acciones que puedan ser ejecutadas.

La dificultad radica en que en una configuración terminal todos los procesos **no están necesariamente en un estado terminal.**

**Estado Terminal**: Un estado en el cual ningún evento se puede  producir en un proceso.

# Detección de Terminación

**Ejemplo**: configuración en la que **todos** los procesos están en un estado que autoriza la recepción de mensajes, pero **no hay mensajes** en tránsito.
En este ejemplo, la aplicación terminó, pero para cada proceso el estado en el cual se encuentra, podría también producirse durante el curso de la ejecución.

En estos casos, se dice que la **terminación es implícita**, ya que los procesos no están concientes de la terminación.

La **terminación es explícita** para un proceso cuando él se encuentra en un estado terminal.

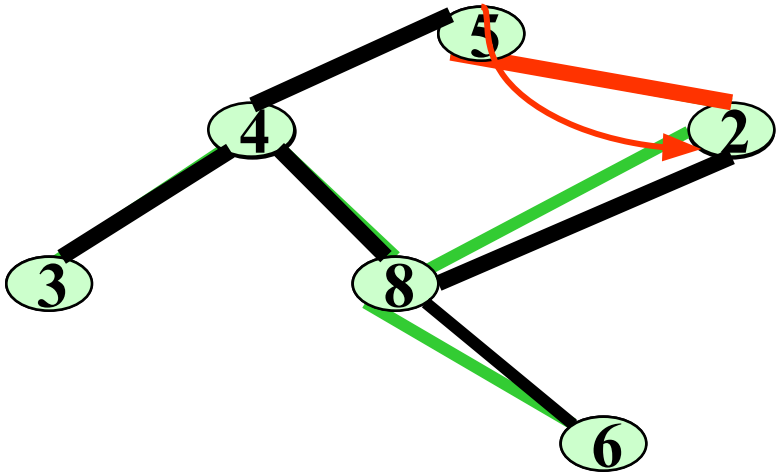
# Ejemplo de Terminación implícita

## Algoritmo de recorrido en paralelo de una red

Se puede probar teóricamente que el algoritmo termina ya que el número total de eventos susceptibles de producirse es limitado.

Sin embargo, esta terminación es inexplotable, ya que no es conocida por ningún nodo.

# Recorrido de redes: Ejemplo Pr = 8




Vecinos(3) = {4}
Vecinos(4) = {3,5,8}
Vecinos(5) = {2,4}
Vecinos(2) = {5,8}
Vecinos(8) = {2,4,6}
Vecinos(6) = {8}

Mensaje Rec

# Ejemplo de Terminación implícita

## Algoritmo de recorrido en paralelo de una red

Se puede probar teóricamente que el algoritmo termina ya que el número total de eventos susceptibles de producirse es limitado.

Sin embargo, esta terminación es inexplotable, ya que no es conocida por ningún nodo.

# Necesidad de terminación explícita

Para que el resultado de una aplicación pueda ser considerado como definitivo, es necesario alcanzar un estado donde la terminación sea explícita.

Es más fácil determinar terminación implícita que terminación explícita, por tanto, el problema se resuelve en dos etapas:

1. Un primer algoritmo observa la aplicación y detecta que alcanza una configuración terminal.
- Un segundo algoritmo difunde esta información a todos los procesos, obligándolos a alcanzar un estado terminal.

# Algoritmo de Huang (1989): Principios

Uno de los procesos monitorea la aplicación y se denomina: *Agente controlador.*

El algoritmo se basa en asignar un peso (W) a cada proceso activo (incluido el controlador) y a cada mensaje en tránsito:

$\forall$ W, $0 \leq W \leq 1$ y satisfacen la invariante $\sum W = 1$.

Inicialmente todos los procesos están en estado pasivo (u ocioso), el peso del controlador es 1 y el del resto de los procesos es 0

El controlador inicia la detección al enviar un *mensaje de la aplicación* a otro proceso.

# Algoritmo de Huang (1989) Principios

Cuando un proceso envía un *mensaje de aplicación* su peso es dividido entre él y el proceso receptor. El peso del proceso receptor es anexado al mensaje

Cuando un proceso recibe un mensaje de aplicación adiciona el peso recibido a su peso.

Cuando un proceso termina o pasa a estado pasivo envía un mensaje de control, traspasando su peso al controlador.

Cuando el peso del controlador vuelve a ser 1 asume que la aplicación terminó.

# Algoritmo de Huang

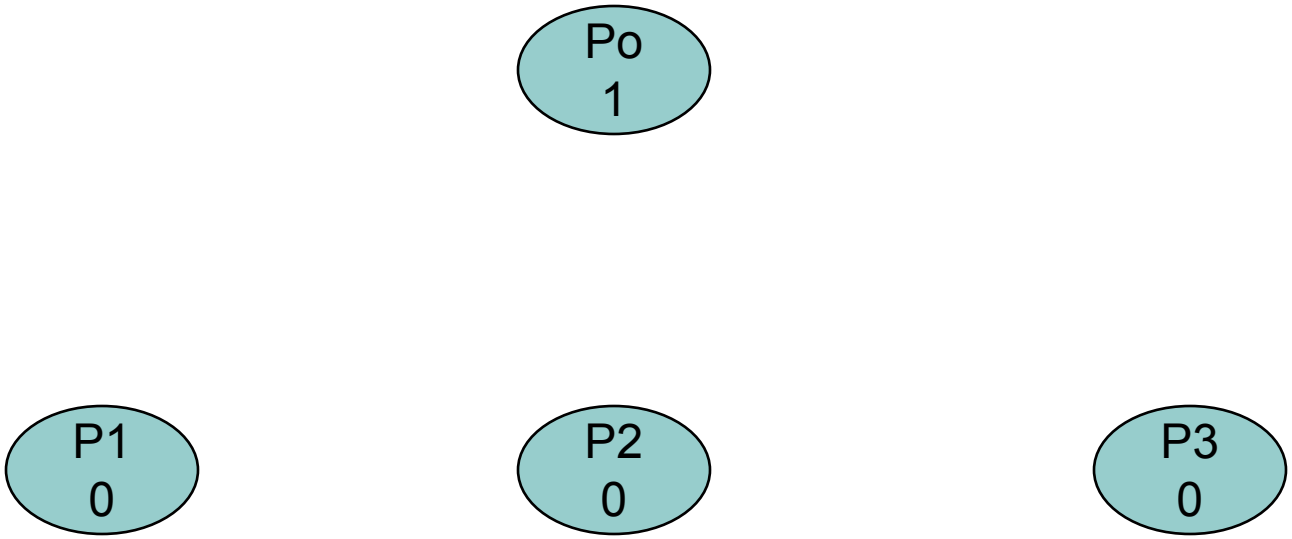
**Mensaje de aplicación:** (mensaje,peso_men)
**Mensaje de control**: (control, peso_men)

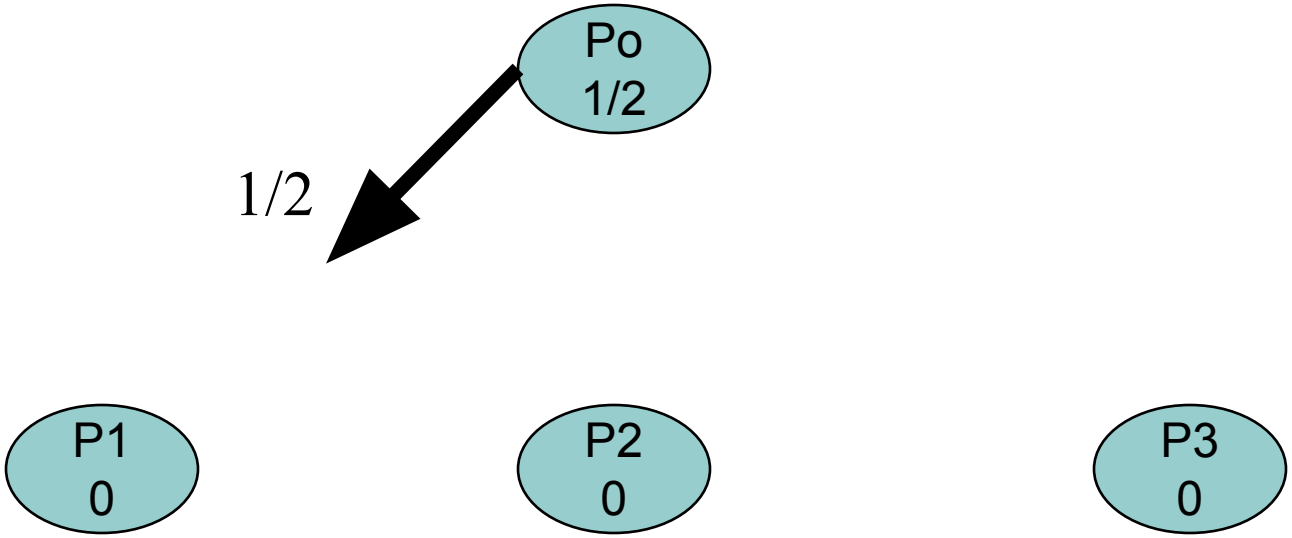
```
Var    peso : Entero inicializa 0;
   estado (activo,pasivo) inicializa pasivo;
% Al  iniciar el algoritmo.
   si Id = Controlador entonces peso:=1;
   fsi;
Cuando (desee enviar (mensaje) a receptor) hacer
Calcular(peso1,peso2) tq peso1 + peso2=peso,
                  peso1>0, Peso2>0.
   peso := peso1;
   enviar(mensaje,peso2) a receptor;
fcuando;
```

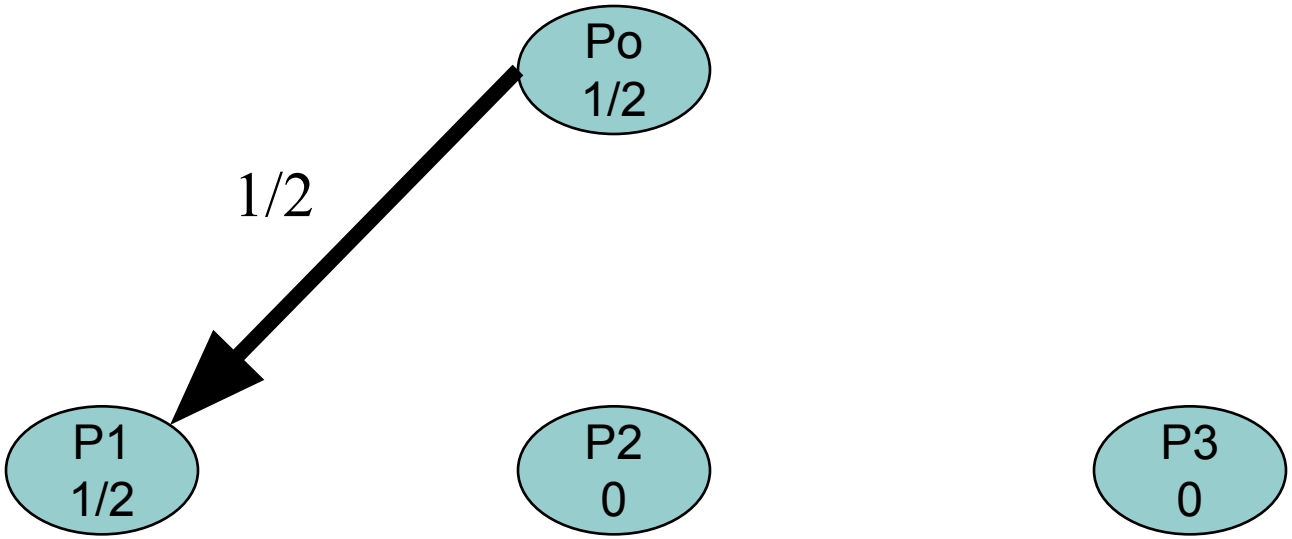
# Algoritmo de Huang

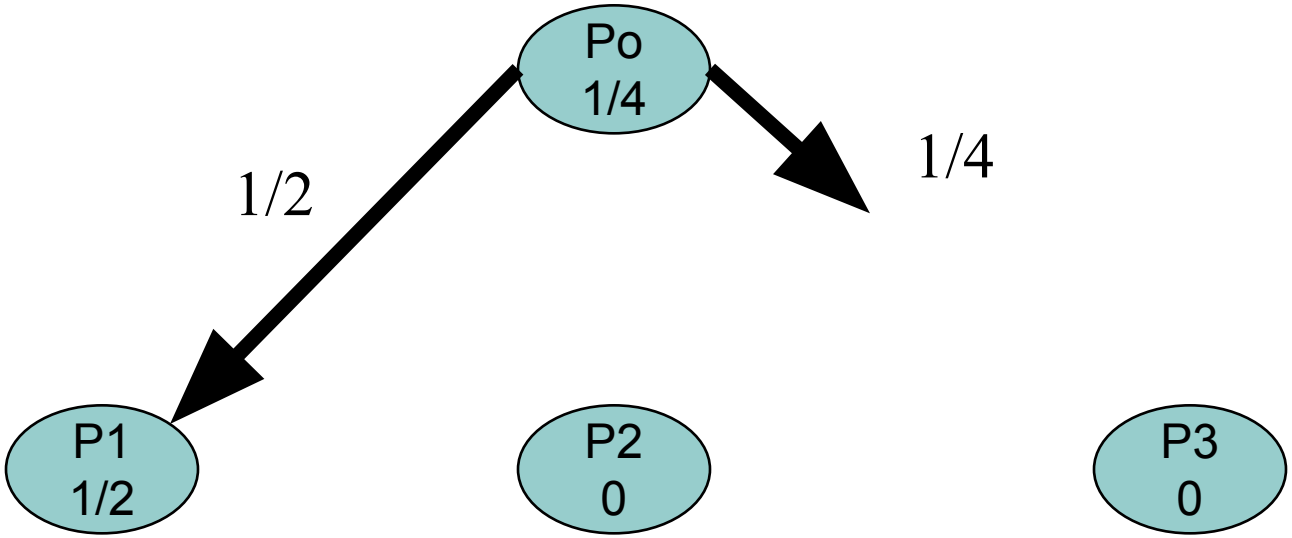
```
Cuando reciba  (mensaje,peso_men) hacer
    peso:=peso + peso_men;
    estado = activo;
fcuando;
Cuando reciba (control,peso_men) hacer
     si Id = controlador entonces
        peso := peso + peso_men;
        Si peso = 1 entonces % terminó la aplicación
        fsi;
    fsi;
fcuando;
Cuando un proceso pasa a estado pasivo hacer
    enviar(control,peso) a controlador;
    estado:=pasivo; peso:=0;
fcuando;
```

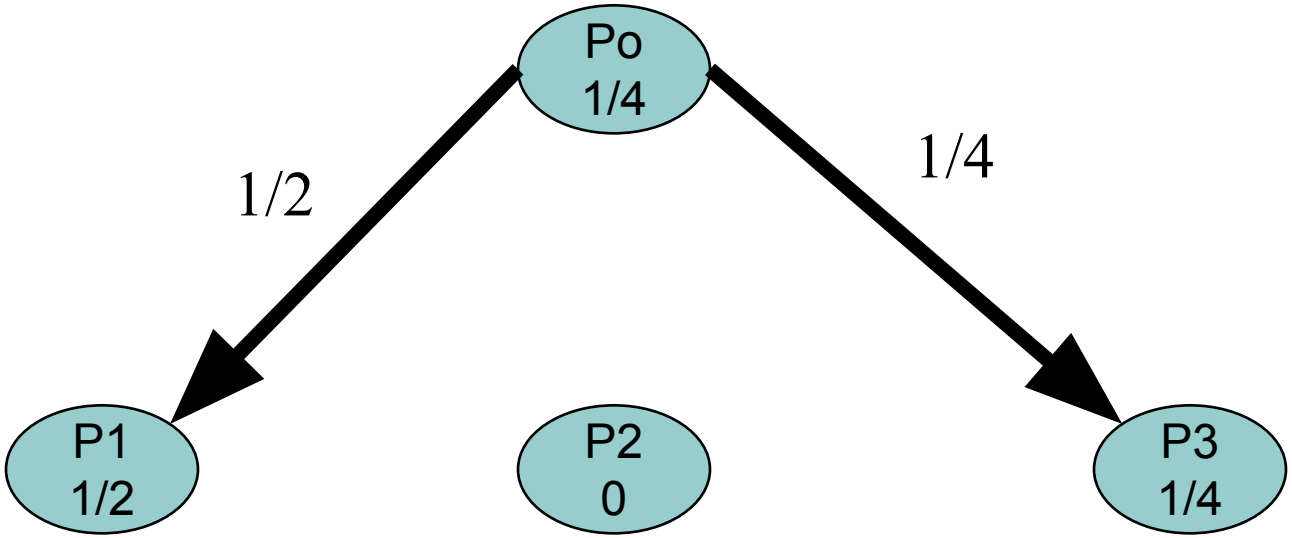
# Algoritmo de Huang: Ejemplo

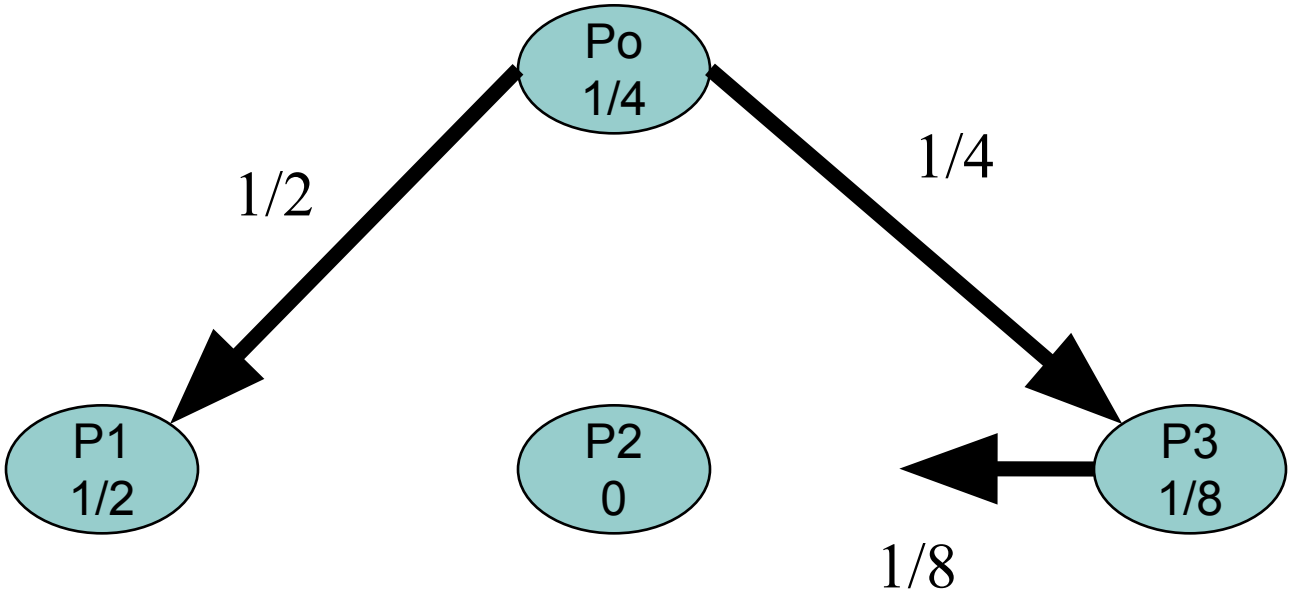


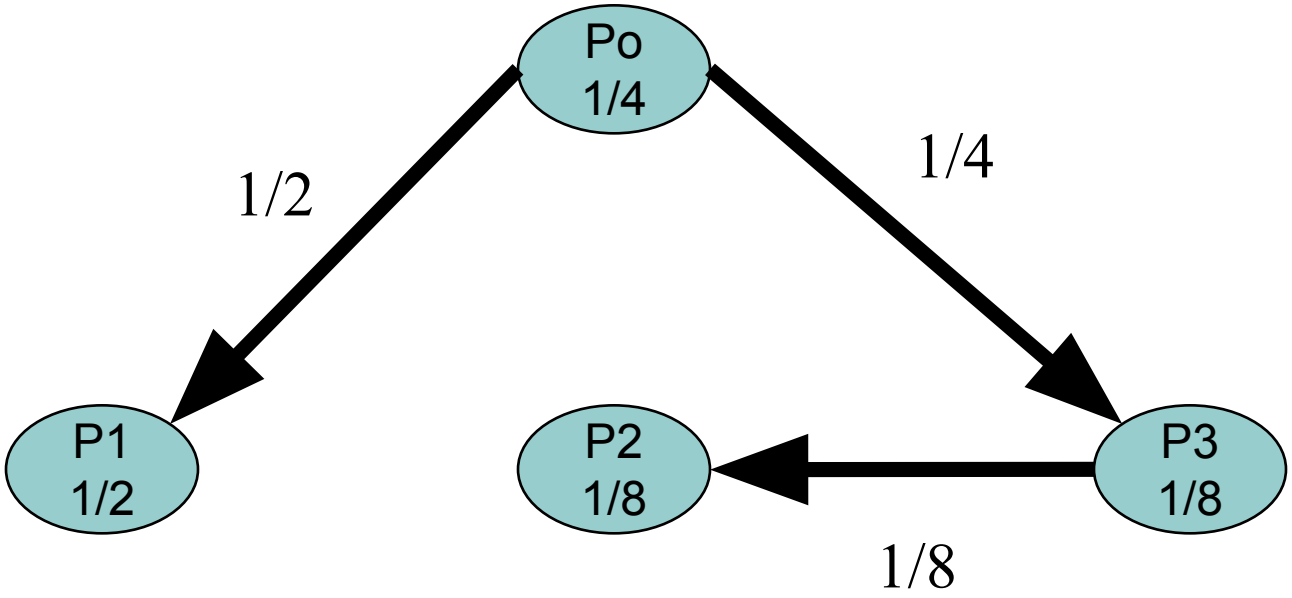

Po
1/2
1/2
P1
0
P2
0
P3
0

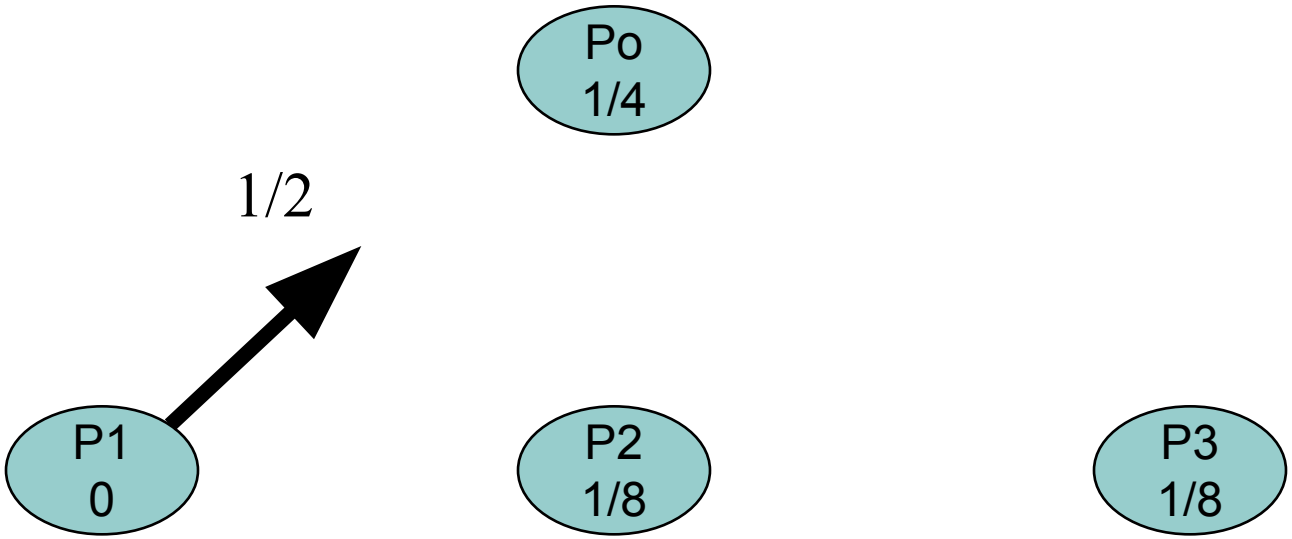

Po
1/2
1/2
P1
1/2
P2
0
P3
0

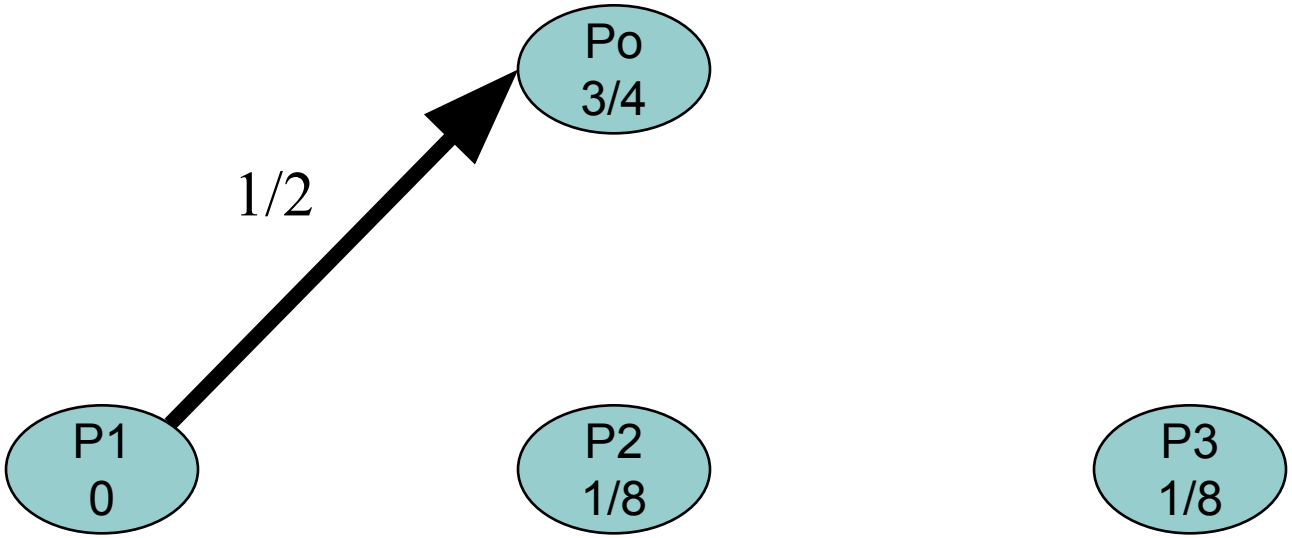

Po
1/4
1/2
1/4
P1
1/2
P2
0
P3
0

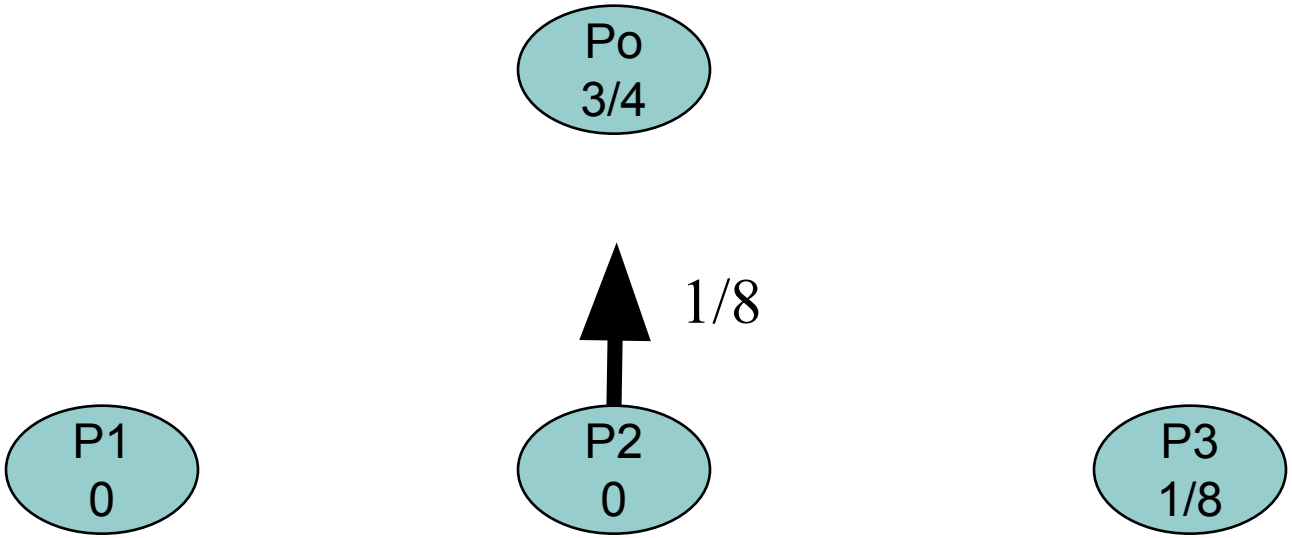

Po
1/4
1/2
1/4
P1
1/2
P2
0
P3
1/4

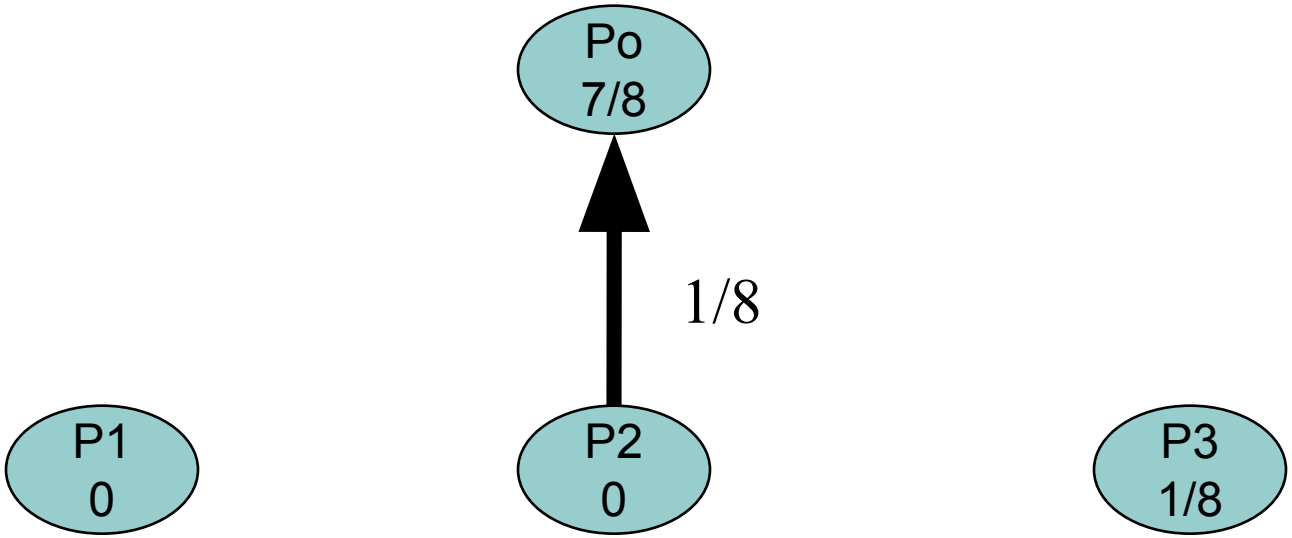

Po
1/4
1/2
1/4
P1
1/2
P2
0
P3
1/8
1/8

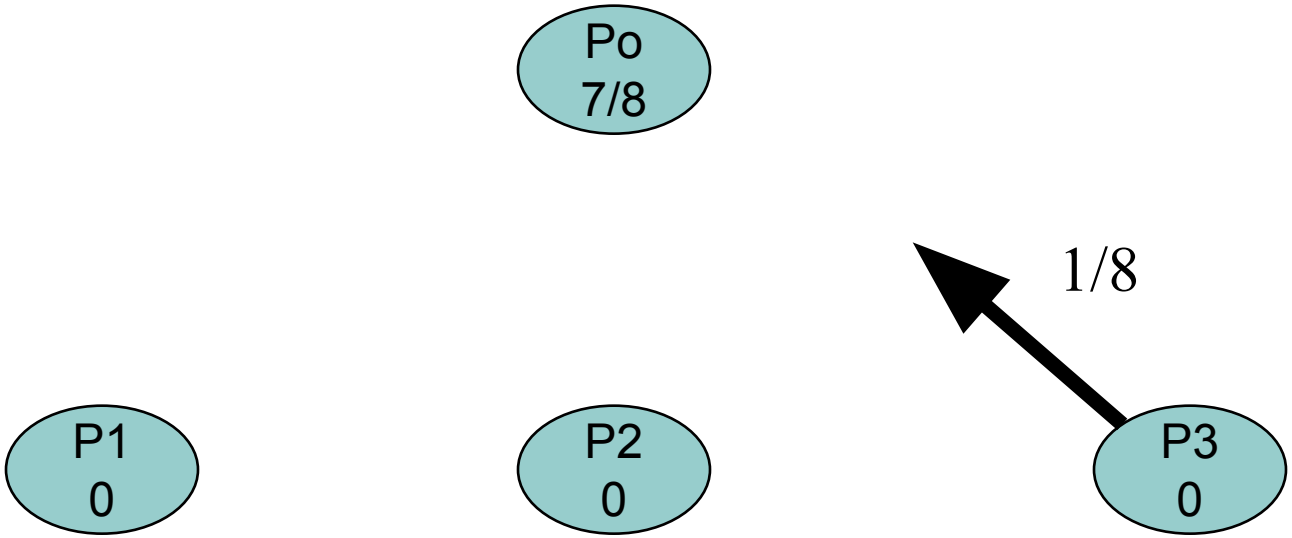

Po
1/4
1/2
1/4
P1
1/2
P2
1/8
P3
1/8
1/8

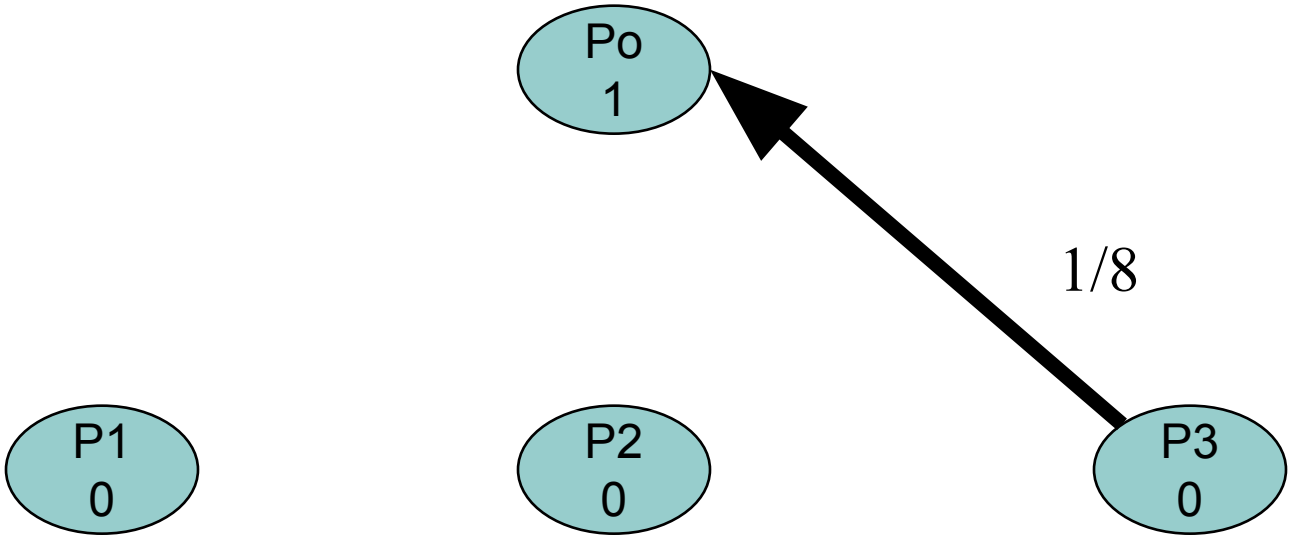

Po
1/4
1/2
P1
0
P2
1/8
P3
1/8

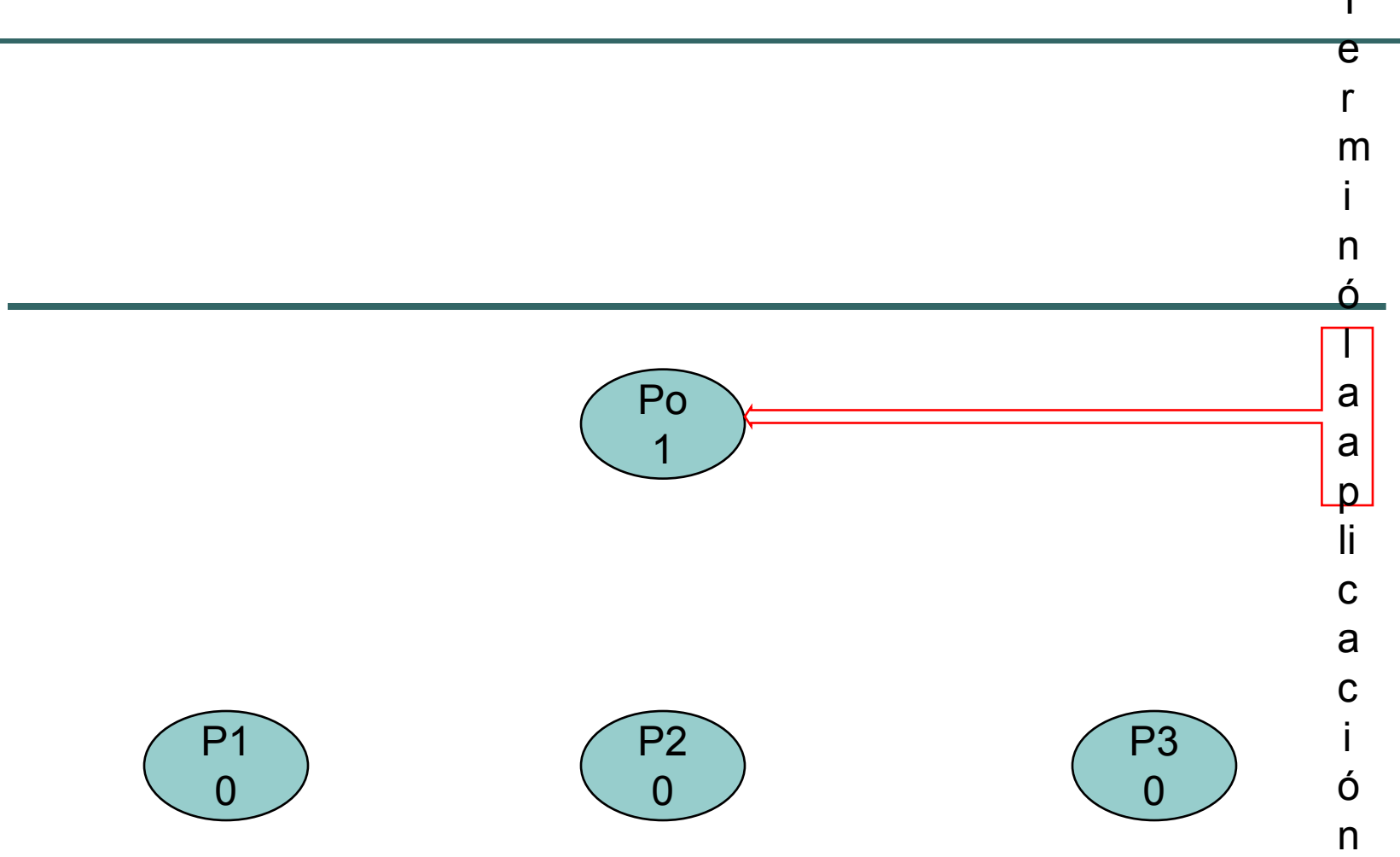

Po
3/4
1/2
P1
0
P2
1/8
P3
1/8


Po
3/4
1/8
P1
0
P2
0
P3
1/8


Po
7/8
1/8
P1
0
P2
0
P3
1/8


Po
7/8
1/8
P1
0
P2
0
P3
0


Po
1
1/8
P1
0
P2
0
P3
0


Terminó la aplicación
Po
1
P1
0
P2
0
P3
0

# Algoritmo de Huang : características

- El control es centralizado
- Supone una topología de estrella para la detección de terminación
- La topología de la aplicación puede ser cualquiera
- Sólo se intercambian mensajes de control cuando los procesos pasan a estado pasivo

# Algoritmo de Dijsktra , Feijen y Van Gasteren: Estados de un Proceso

- Estado activo : Un proceso se encuentra en estado activo cuando lleva a cabo un evento interno ó envía un mensaje . En cualquier otro caso es pasivo .
- Estado pasivo : Un proceso se encuentra en estado pasivo si está en espera de un mensaje ó si no puede ocurrir ningún evento interno , en cuyo caso se encuentra en estado terminal. En cualquier otro caso es activo .

```
Var estado_P = (activo , pasivo);
      Al recibir un mensaje
          Recibir (aplicación , mensaje) de q ;
          estado_P = activo ;
       Para enviar un mensaje de p a q
          Si estado_P == activo entonces
       enviar (aplicación , mensaje) a q ;
          Fsi
```

```
Si el proceso P termina ó
está a la espera de un mensaje
          Si estado_P == activo entonces
                estado_P = pasivo ;
           Fsi
```

# Principios

- El proceso que inicia la detección envía un mensaje de control, llamado **token** , sobre un anillo lógico que contiene a todos los procesos. Una vez que el **token** regresa al proceso iniciador este será el encargado de determinar si existe terminación o no.
- El anillo lógico sobre el que se establece este algoritmo es independiente de la topología lógica utilizada por la aplicación de usuario sobre la cual se realizara la detección.
- El anillo lógico contendrá los **N** procesos involucrados, enumerados del **0** hasta **N-1**, donde el proceso $P_0$ es el encargado de iniciar el algoritmo enviando el **token** al proceso $P_{N-1}$ , este a su vez envía el **token** al proceso $P_{N-2}$ , y asi sucesivamente hasta que el proceso $P_1$ envía el **token** al proceso $P_0$ y éste realiza la detección.

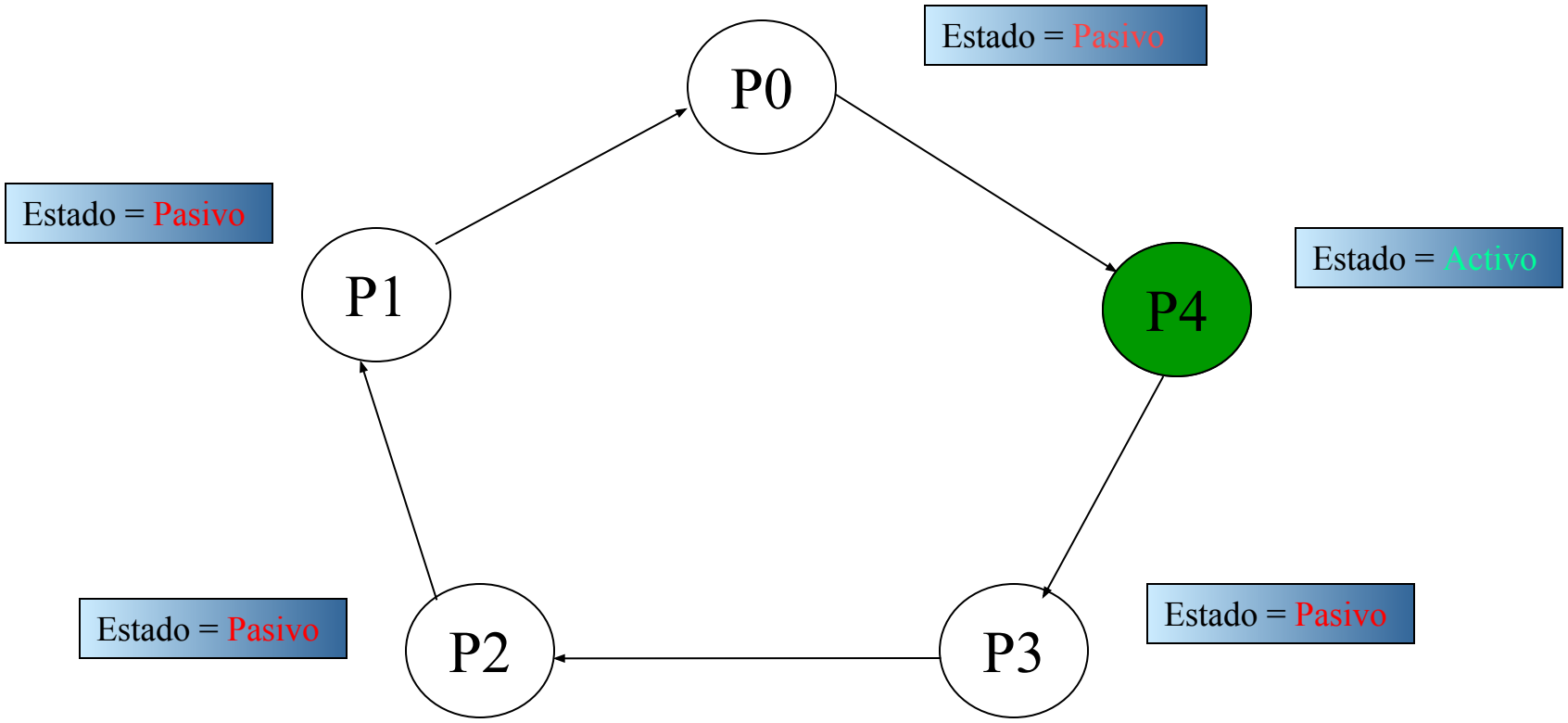
# Hipótesis

- El pase de mensajes es síncrono .
- No hay pérdidas de mensajes .
- La comunicación que se establece sobre el anillo lógico es uni-direccional .
- El proceso $P_0$ es el encargado de la detección .
- La terminación está caracterizada por el estado del sistema donde todos los procesos están en estado pasivo y las líneas de comunicación entre los procesos están vacías .

# Reglas

- Regla 1: Cuando esté Activo, un proceso $P_{i+1}$ guarda el token . Cuando pasa a estado Pasivo lo transmite al proceso $P_i$ .

# Falsificación de la Regla 1




Fals a Terminación

# Reglas Básicas

- Regla 1: Cuando esté Activo, un proceso Pi+1 guarda el token. Cuando pasa a estado pasivo lo transmite al proceso Pi.
- Regla 2: Cada proceso independientemente de su estado tendrá asociado un color (**Blanco**, **Negro**). Inicialmente todos estarán en **Blanco**.
- Regla 3: El proceso que envía un mensaje de aplicación se colorea **Negro** el mismo .
- Regla 4: El token tendrá asociado un color (**Blanco, Negro**) .
- Regla 5: Cuado el proceso Pi+1 propaga el token, si el proceso Pi+1 es **Negro** envía un token **Negro** al proceso Pi. Si el proceso Pi+1 es **Blanco**, él no cambia el color del token.
- Regla 6: Cada proceso se convierte en **Blanco** inmediatamente después de enviar el token

# Reglas Complementarias

- Regla 7: El proceso $P_0$ inicia una encuesta, enviando un **token Blanco** al proceso $P_{n-1}$ .
- Regla 8: Si el **token** regresa **Blanco** a $P_0$ , y $P_0$ en ese instante es **Blanco** y **Pasivo**, se concluye la terminación de los procesos. En cualquier otro caso la encuesta es infructuosa.
- Regla 9: Después del retorno de una encuesta infructuosa, el proceso $P_0$ inicia una nueva encuesta .

# Algoritmo [DFvG 83]

```
Var estado_P = (activo , pasivo);
      color_P = (blanco , negro) inicializa blanco ; // (Regla 2)
      tock = (blanco , negro) ;     // (Regla 4 )

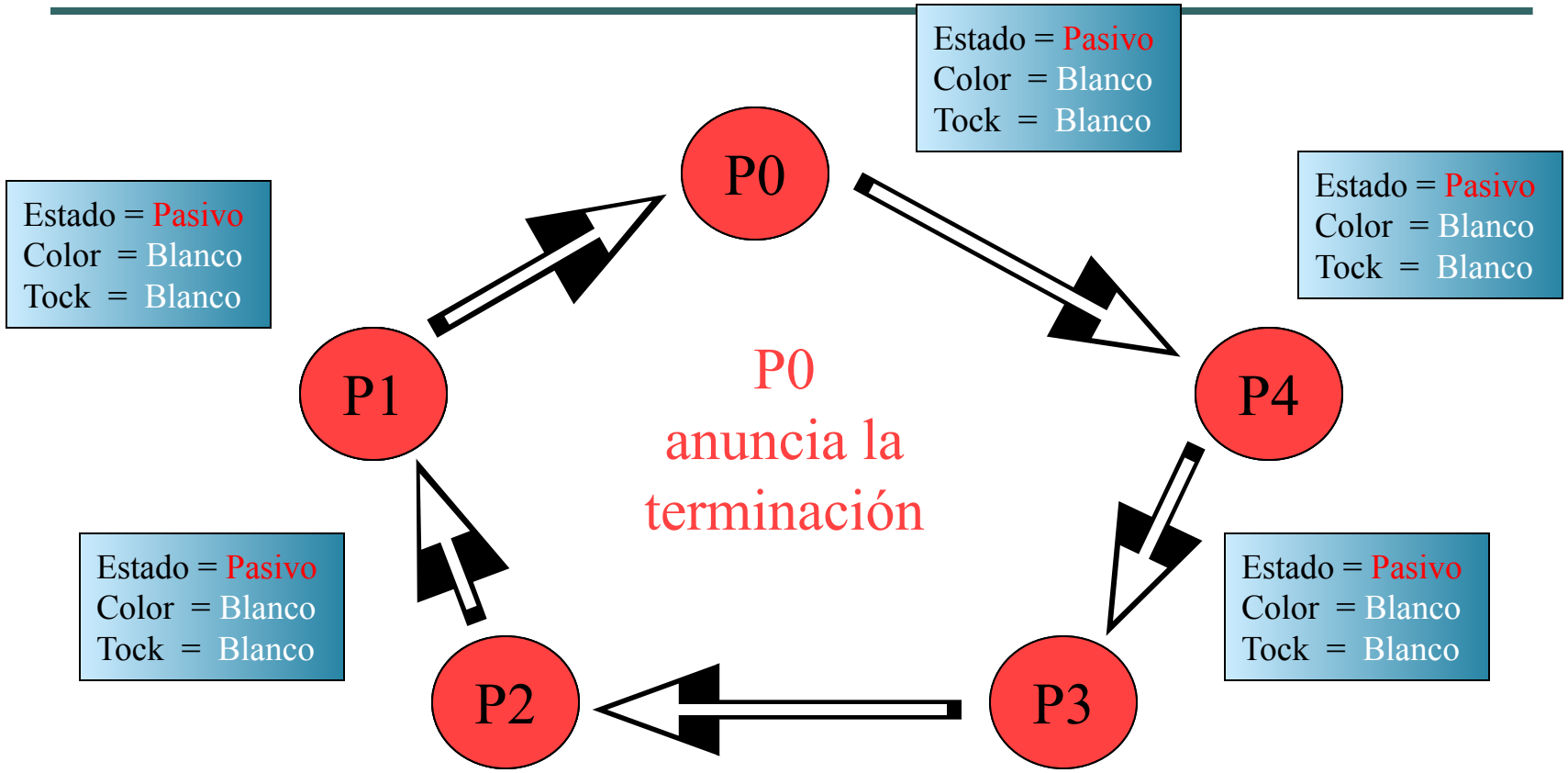
      Al recibir un mensaje de aplicación de q i
      {     Recibir (aplicación , mensaje) de  * ;
            estado_P = activo ;
      }
      Para enviar un mensaje de aplicación de p a q
      {     Si estado_P == activo entonces
                  color_P = negro ;   // (Regla 3)
                  enviar (aplicación , mensaje) a q ;
            Fsi
       }
       Si el proceso P termina ó está a la espera de un mensaje
       {    Si estado_P == activo entonces  estado_P = pasivo ;
            Fsi
       }
```

# Algoritmo [DFvG 83]

```
Si  (ID == 0)  entonces      //   Para iniciar la detección por parte de P0
     Enviar ( token , blanco) a P N-1 ;    // (Regla 7)
Fsi

//   Cuando llegue mensaje tipo Token
Recibir ( token , tock) de  * ;
Cuando (estado_P == pasivo) hacer
          Si  (ID == 0)  entonces
                    Si ((color_P == blanco) and (tock == blanco)) entonces
               anunciar_terminación() ;     // (Regla 8)
                SiNo        Enviar ( token  , blanco) a P N-1 ;   // (Regla 9)
                Fsi
                  SiNo
                       Si (color_P == blanco) entonces
                 Enviar ( token  , tock) a P ID-1 ;    // (Regla 5)
                SiNo  Enviar ( token  , negro) a P ID-1 ;        // (Regla 5)
                Fsi
           Fsi
             Fsi
            color_p = blanco ;    // (Regla 6)
  Fcuando
```

# Algoritmo [DFvG 83]




Detección de Terminación

# Características Principales

- El proceso iniciador es el encargado de decretar la terminación, por lo tanto el algoritmo depende de dicho proceso.
- La comunicación entre procesos es síncrona.
- Topología de anillo.

